Elephant-Goldfish Model: Prompting Playbook

Oct 7, 2026 · @Remy

Distilled from Dave Rensin's article "Elephants, Goldfish and the New Golden Age of Software Engineering" (April 2026). The core move: make the AI interrogate you before it answers, and test what you write against a session with no memory.

Core ideas

1. **Interrogate, don't instruct.** Most bad output comes from a thin problem description. Ask the model to keep questioning you and challenging assumptions until you say stop.
2. **Define success before the work.** Build acceptance criteria in a separate pass, then hand the task and the criteria to the model together.
3. **Break sycophancy on purpose.** When it starts agreeing, tell it that agreement isn't help and challenge is. "Why do you think that?" often ends a hallucination loop.
4. **Make the model propose first.** If you propose and it reacts, you never learn whether it actually understands the problem.
5. **Docs are cheaper context than code.** A design doc re-bootstraps a fresh session in seconds; raw code is slow and token-heavy.
6. **Design is the new code.** When AI writes the code, the "why" behind each micro-decision must move into the design doc, or the system becomes opaque to you.
7. **Test docs with a blank session.** If a fresh session can't understand the doc alone, it depends on chat history it doesn't contain.
8. **You own the judgment.** The model produces text that resembles judgment; the decisions stay yours.
9. **Save artifacts, not transcripts.** Keep the problem description, acceptance criteria, and final output. Chat logs are mostly noise.

Examples from the article

Example	What happened	Lesson
Buying a car	One research prompt produced an impressive but shallow report	Spend ~20 min being questioned, ~20 min building a grading rubric with a skeptical fact-checker persona, then run deep research in a fresh session with both
Aha #1: the crash	A session died an hour in and all context was lost	A short doc of intent and progress makes re-bootstrapping a 30-second step
Aha #2: off the rails	With context restored, the model invented APIs and made bad architecture choices	Context isn't enough; the model needs strict guardrails
Aha #3: docs over code	Feeding design docs instead of source gave a fresh session enough understanding	Docs are far smaller and more reliable context than code
The internal tool	Dozens of contributors added features to a tool used by 100k+ people	The process keeps quality and security bars at high speed
Large codebases	AI wrote a README.md per directory, leaves up to root; ~50% wrong at the leaves, fixed in 5–10 min each	Recursive summaries bootstrap a monolith in about a week
Onboarding	New contributors get a notebook of READMEs and docs, and must write a doc that passes the fresh-session test	Passing the test shows they understand enough to contribute

The Elephant-Goldfish format

Feed the Elephant, test against the Goldfish: one long session builds the design doc, and fresh sessions check it before any code is written.

Roles

- **Elephant:** one long, context-rich session. It produces the design doc.
- **Goldfish:** a new session with no history. It sees only the doc and the files the doc points to.
- **Human:** owns the judgment, settles arguments, approves the doc.

Phases

1. **Load context** (skip for a new project). Point the model at existing docs or the source tree and ask it to summarize what it understands. Correct it immediately. With no docs, build READMEs from the deepest folders up to the root first.
2. **Define the problem, no code.** Tell the model not to write code and to question you instead. Argue for 20–30 minutes; when it's wrong, point it to the file that shows the truth. Output: a few pages of problem description.
3. **Agree on a design.** Ask the model to propose an implementation in prose and block diagrams (short pseudocode is fine). Ask why it chose each pattern, correct it, repeat. This can take hours or days.
4. **Write the design doc, one section at a time** in the same session: Problem (3–5 sentences), Technical plan, Alternatives considered, Detailed implementation listing every file and why, Acceptance criteria. Save it beside the code.
5. **Test against fresh sessions.** Run each test in the table below in a new session; update the doc after each failure and rerun.
6. **Implement.** Give a new session the doc and tell it to follow the plan exactly. If it crashes or drifts, start another with the doc.
7. **Harsh review.** Tell the model you suspect the code is poor and want it torn apart; flag long uncommented stretches. Repeat until the critique is trivial.
8. **Save.** Keep the doc, criteria, and output; discard the chat logs.

Test	Ask the fresh session	Passes when
Comprehension	What is this doc trying to do, and how does the current system relate to it?	It explains both correctly from the doc alone
Critic	As an expert reviewer, find gaps, wrong assumptions, and missed edge cases	Remaining feedback is only nitpicks
Readiness	Could you implement this correctly on the first pass from this doc?	No important questions remain

Test	Ask the fresh session	Passes when
Human review	A real person reads it	They approve, or you use your own judgment if solo

Design doc skeleton

Copy this into your repo as the starting point for each feature's doc.

```
# <Feature name> — Design Doc
Status: draft | goldfish-tested | approved | implemented

## 1. Problem
<3–5 sentences a casual reader understands>

## 2. Technical Plan
<components, how they fit, block diagram>

## 3. Alternatives Considered
- <Option> — rejected because <reason>

## 4. Detailed Implementation
| File | Create/Change | What | Why |
|------|---------------|------|-----|

Steps:
1. ...

## 5. Acceptance Criteria
- <testable condition>

## 6. Open Questions / Decisions Log
- <date> — <decision> — <rationale>
```

Caveats

- **Scale it to the task.** The full process pays off for multi-file features and long-lived codebases. For a short script or a homework bug fix, the problem-definition phase plus a harsh review is usually enough.

- **A doc a fresh session understands isn't necessarily correct.** The test shows the doc stands on its own. It can still be clear, consistent, and wrong, so human review and real tests still matter.
- **Expect a low critic hit rate.** The article puts useful suggestions at about 30%. Deciding which to keep is the judgment part.
- **Keep the doc in sync.** If the code changes and the doc doesn't, future sessions work from a stale source of truth. Update the doc whenever the plan shifts during implementation.
- **Acceptance criteria are an addition.** The article's design doc has four sections; section 5 brings in the grading rubric from its car-buying example.